

ARTIFICIAL INTELLIGENCE

Submitted By :

Utsav Mittal (102306005)

3f11



**DEPARTMENT OF ELECTRONICS AND COMMUNICATION
ENGINEERING**

Thapar Institute of Engineering & Technology , Patiala

AI ML-LAB (UNC503)

Assignment 1: Fundamentals of Python Programming

S. No.	Experiments
1	Number Data Types and Arithmetic Operations Develop a program demonstrating various numeric data types and performing arithmetic operations including addition, subtraction, multiplication, division, and exponentiation.
2	Boolean Logic Operators Implement a program that illustrates the use of common Boolean operators: and, or, and not, with clear examples.
3	String Manipulations Write a program to perform the following string operations: • Display a string • Concatenate two strings • Capitalize a string • Convert a string to uppercase • Right-justify and center-align a string • Replace all occurrences of a substring with another • - Extract a substring from a given string
4	List Operations Create a program to handle a list containing diverse data types: • Access specific elements • Slice the list • Append new elements and remove existing ones
5	Set Operations Develop a program to work with sets containing various data types: • Check if an element exists in the set • Add and remove elements
6	Tuple Usage Create a program demonstrating tuples with different data types and use tuples as keys to create a dictionary.
7	Dictionary Manipulations Write a program to create a dictionary with mixed data types: • Access values using keys • Add new key-value pairs and remove existing ones
8	Write a program that prints whether a given number is positive, negative, or zero.
9	Implementation of ReLU Function Implement the Rectified Linear Unit (ReLU) function using conditional statements, defined as: $relu(x) = \begin{cases} x, & \text{if } x \geq 0, \\ 0, & \text{otherwise} \end{cases}$
10	Advanced Programming Tasks a) Write a Python program to print all prime numbers less than 20. b) Implement a recursive function to compute the factorial of a number. c) Develop a program that reads a text file, extracts all unique words, and prints them in alphabetical order.
11	Comprehension Create Python programs using comprehensions to: • Square each element in a list using list comprehension. • Square each value in a dictionary using dictionary comprehension.

CODE1:

```
1 A=10
2 B=3
3 print("addition",A+B)
4 print("substraction",A-B)
5 print("multiply",A*B)
6 print("division",A/B)
7 print("exponential",A**B)
8
9
```

OUTPUT:

```
addition 13
substraction 7
multiply 30
division 3.3333333333333335
exponential 1000
```

CODE2:

```
11 x=True
12 y=False
13 print("x and y",x and y)
14 print("x and y",x or y)
15 print("not x", not x)
```

OUTPUT:

```
x and y False
x and y True
not x False
```

CODE3:

```
hello"
python"
print("display string:",S1)
print("CONCATENATION:",S1+" "+S2)
print("capitalised:",S1.capitalize())
print("uppercase:",S1.upper())
print("right justified:",S1.rjust(10))
print("center aligned:",S1.center(10))
print("replace:",S1.replace("l","x"))
print("substring:",S1[1:4])
```

OUTPUT:

Output	Clear
display string: hello	
CONCATENATION: hello python	
capitalised: Hello	
uppercase: HELLO	
right justified: hello	
center aligned: hello	
substring: ell	

CODE4:

```
32 lst=[1,"apple",3.5,True]
33 print("access element:",lst[1])
34 print("slice list:",lst[1:3])
35 lst.append("new")
36 print("after append:",lst)
37 lst.remove("apple")
38 print("after remove:",lst)
39
40
```

OUTPUT:

Output

Clear

```
access element: apple
slice list: ['apple', 3.5]
after append: [1, 'apple', 3.5, True, 'new']
after remove: [1, 3.5, True, 'new']
```

```
=== Code Execution Successful ===
```

CODE5:

```
41 s={1,2,3,"a"}
42 print("check elemnt:",2 in s)
43 s.add(5)
44 print("after add",s)
45 s.remove(1)
46 print("after remove",s)
```

OUTPUT:

Output

Clear

```
check element: True
after add {1, 2, 3, 5, 'a'}
after remove {2, 3, 5, 'a'}
```

=== Code Execution Successful ===

CODE7:

```
50 t=(1,"data",3.14)
51 print("tuple",t)
52 d={t:"tuple as key"}
53 print("dictionary",d)
54
55
```

OUTPUT:

Output

Clear

```
tuple (1, 'data', 3.14)
dictionary {(1, 'data', 3.14): 'tuple as key'}
```

```
=== Code Execution Successful ===
```


CODE8:

```
56
57 d={"name":"alice","age":20,"marks":85}
58 print("access value:",d["name"])
59 d["gerade"]="A"
60 print("after adding:",d)
61 d.pop("age")
62 print("after removing:",d)
```

OUTPUT:

Output

Clear

```
access value: alice
after adding: {'name': 'alice', 'age': 20, 'marks': 85, 'gerade': 'A'}
after removing: {'name': 'alice', 'marks': 85, 'gerade': 'A'}
```

=== Code Execution Successful ===

CODE9:

```
67 num = float(input("Enter a number: "))
68
69 if num > 0:
70     print("Positive number")
71 elif num < 0:
72     print("Negative number")
73 else:
74     print("Zero")
75
76
```

OUTPUT

Output

Clear

Enter a number: 2

Positive number

=== Code Execution Successful ===

CODE10:

```
78 ▾ def relu(x):  
79 ▾     if x >= 0:  
80     |         return x  
81 ▾     else:  
82     |         return 0  
83  
84 x = int(input("Enter value: "))  
85 print("ReLU output:", relu(x))
```

OUTPUT:

Output

Clear

Enter value: 33

ReLU output: 33

=== Code Execution Successful ===

CODE10(a)

```
88 ▾ for num in range(2, 20):  
89 ▾     for i in range(2, num):  
90 ▾         if num % i == 0:  
91 ▾             break  
92 ▾     else:  
93 ▾         print(num)  
94 ▾
```

OUTPUT

Output	Clear
2	
3	
5	
7	
11	
13	
17	
19	
=== Code Execution Successful ===	

CODE 10(b)

```
97 ▾ def factorial(n):  
98 ▾     if n == 0 or n == 1:  
99 ▾         return 1  
100 ▾     else:  
101 ▾         return n * factorial(n - 1)  
102  
103 n = int(input("Enter number: "))  
104 print("Factorial:", factorial(n))
```

OUTPUT:

Output

Clear

Enter number: 44

Factorial: 2658271574788448768043625811014615890319638528000000000

=== Code Execution Successful ===

CODE10(c)

```
108 try:
109     with open("sample.txt", "r") as file:
110         words = file.read().lower().split()
111         unique_words = sorted(set(words))
112         print(unique_words)
113 except FileNotFoundError:
114     print("Error: sample.txt file not found")
```

OUTPUT:

Output

Clear

ERROR!

Error: sample.txt file not found

=== Code Execution Successful ===

CODE11(a)

```
118 lst=[1,2,3,4]
119 squared_lst=[x**2 for x in lst]
120 print(squared_lst)
121
```

OUTPUT:

Output

Clear

```
[1, 4, 9, 16]
```

```
=== Code Execution Successful ===
```

CODE 11 (b)

```
122
123 d={"a":1,"b":2,"c":3}
124 squared_dict={k:v**2 for k, v in d.items()}
125 print(squared_dict)
```

OUTPUT:

Output

Clear

```
{'a': 1, 'b': 4, 'c': 9}
```

```
=== Code Execution Successful ===
```